

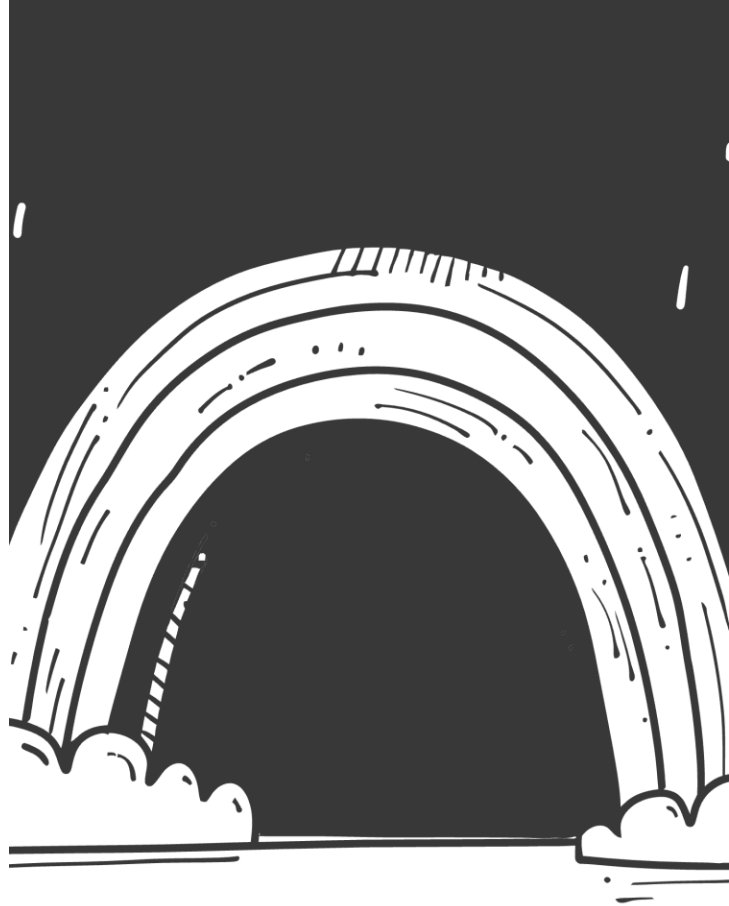
Beyond UI: Rest Assured, Performance & Security

Session 7 · Playwright Automation Bootcamp

REST ASSURED

GATLING

OWASP



What You Will Be Able to Do

01

Read Rest Assured Syntax

Understand RequestSpecification and ResponseSpecification to reduce duplication.

03

Identify OWASP Top 10

Recognize key vulnerabilities and integrate DAST scanning into a CI/CD pipeline.

02

Explain NFR Concepts

Throughput, latency, concurrency, and error rate — and when to use Gatling vs. Karate.

04

Unify the Strategy

Map Playwright, Rest Assured, and performance/security tooling into one cohesive approach.

Two Tools, Two Stacks — One Team

Enterprise teams often run mixed stacks: **TypeScript/Node.js** on the frontend, **Java** on the backend. Rest Assured is the de facto standard for Java API testing — the same way Playwright is for TypeScript.

Aspect	Rest Assured (Java)	Playwright (TypeScript)
Style	Fluent DSL	Async / await
Assertions	Hamcrest matchers	expect()
Schema validation	JSON Schema validator	ajv
Best for	Java API stacks	Node.js & browser+API

Fluent API Testing in Java

Rest Assured uses a **Given / When / Then** DSL that mirrors BDD language and maps directly to HTTP request structure.

GET Request

```
given()
    .baseUrl("http://localhost:3000")
    .header("Authorization", "Bearer " + token)
.when()
    .get("/api/products/1")
.then()
    .statusCode(200)
    .body("product.name",
        equalTo("Wireless Noise-Cancelling Headphones"))
    .body("product.price", greaterThan(0.0f));
```

POST with Body

```
given()
    .baseUrl("http://localhost:3000")
    .header("Authorization", "Bearer " + adminToken)
    .contentType(ContentType.JSON)
    .body("{ \"name\": \"New Product\",
        \"price\": 29.99 }")
.when()
    .post("/api/products")
.then()
    .statusCode(201)
    .body("product.id", notNullValue());
```

Define Once, Reuse Everywhere

Without Specifications

Every test repeats `baseUrl`, `headers`, and `content type` — fragile and verbose.

```
given()
    .baseUrl("http://localhost:3000")
    .header("Authorization", "Bearer " + token)
    .contentType(MediaType.JSON)
.when().get("/api/products")
.then().statusCode(200);
```

With Specifications

Define once in `@BeforeAll`, every test becomes a single concise line.

```
RequestSpecification requestSpec =
    new RequestSpecBuilder()
        .setBaseUrl("http://localhost:3000")
        .addHeader("Authorization", "Bearer " + token)
        .setContentType(MediaType.JSON)
        .build();

// Every test is now concise
given(requestSpec)
    .when().get("/api/products")
    .then().spec(responseSpec);
```

Schema Contracts & API Chaining

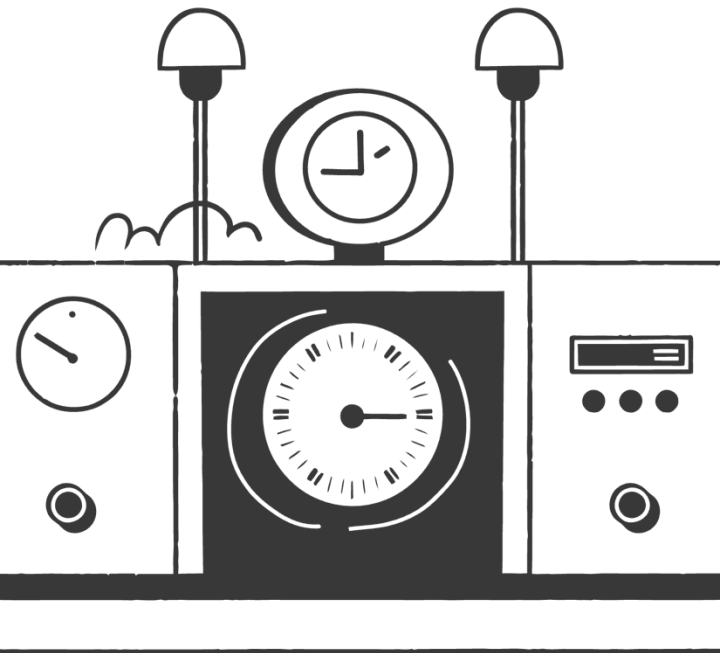
JSON Schema Validation

```
given(requestSpec)
  .when().get("/api/products/1")
  .then()
    .statusCode(200)
    .body(matchesJsonSchemaInClasspath(
      "schemas/product-schema.json"));
```

API Chaining — Login → Create

```
// Step 1: extract token
String token =
  given().contentType(ContentType.JSON)
    .body("{ \"email\": \"admin@example.com\",
      \"password\": \"Admin123!\" }")
    .when().post("/api/auth/login")
    .then().statusCode(200)
    .extract().path("token");

// Step 2: use token
given()
  .header("Authorization", "Bearer " + token)
  .body("{ \"name\": \"Lab Product\" }")
  .when().post("/api/products")
  .then().statusCode(201)
    .extract().path("product.id");
```



Shift Left on Performance

Before it hits production — a mindset, a methodology, and a toolkit for catching performance regressions in CI before they cost you customers.

The Performance Vocabulary

Response Time

End-to-end time per request.

Target: p95 < 500ms

Throughput

Requests handled per second.

Target: 500 RPS

Concurrency

Simultaneous active users.

Target: 200 users

Error Rate

Percentage of failed requests.

Target: < 0.1%

Why percentiles matter: A p95 of 800ms means 5% of real users wait longer than 800ms. An average of 200ms can hide a p99 of 5 seconds.



Load, Stress & Spike Tests

Load Test

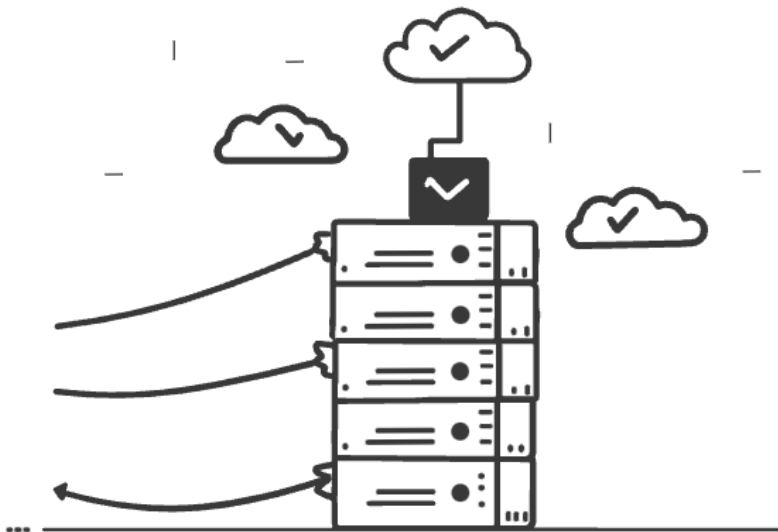
Sustained expected traffic — does the system handle normal usage?

Stress Test

Traffic beyond capacity — at what point does it break?

Spike Test

Sudden burst — does the system recover when traffic normalises?



Functional vs. Performance Mindset

Functional Mindset

- Does it work?
- One user, one request
- Pass / fail
- Test before release
- Fix when it breaks

Performance Mindset

- Does it work **under load**?
- 200 concurrent users, sustained
- Percentiles — p50, p95, p99
- Test on every CI run
- Baseline now, detect regression later

Why Performance Bugs Are Dangerous



Silent Failures

They don't throw exceptions — they just slow down silently, invisible in development where one developer equals one user.

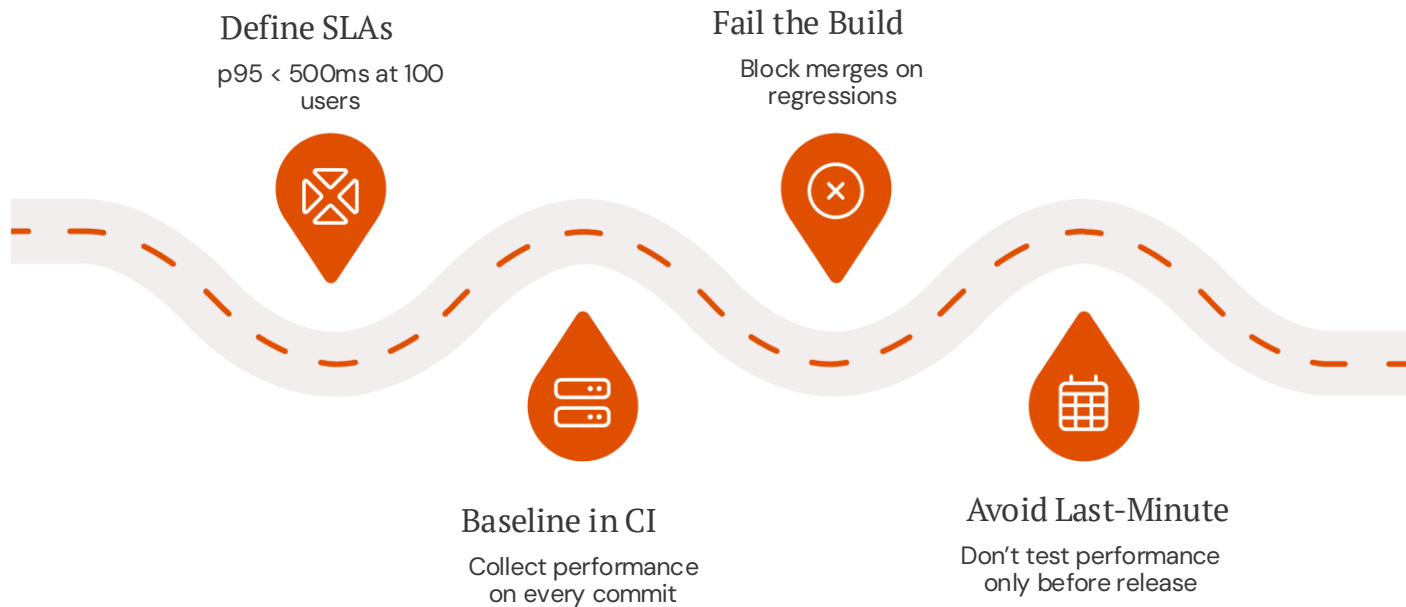
Compound Under Concurrency

A 400ms database query becomes a timeout when 100 users hit it simultaneously.

Cost of Late Discovery

Finding a regression in CI costs hours. Finding it in production costs **customers**.

The Shift-Left Principle



Embed performance gates into your development workflow — not as a last-minute check, but as a continuous quality signal on every commit.

Load Profile Shapes

Different shapes answer different questions about your system's behavior.



Ramp Up

How does the system behave as load increases? Where does it start degrading?



Spike

Does the system recover after a sudden traffic burst? Does error rate return to baseline?



Steady State

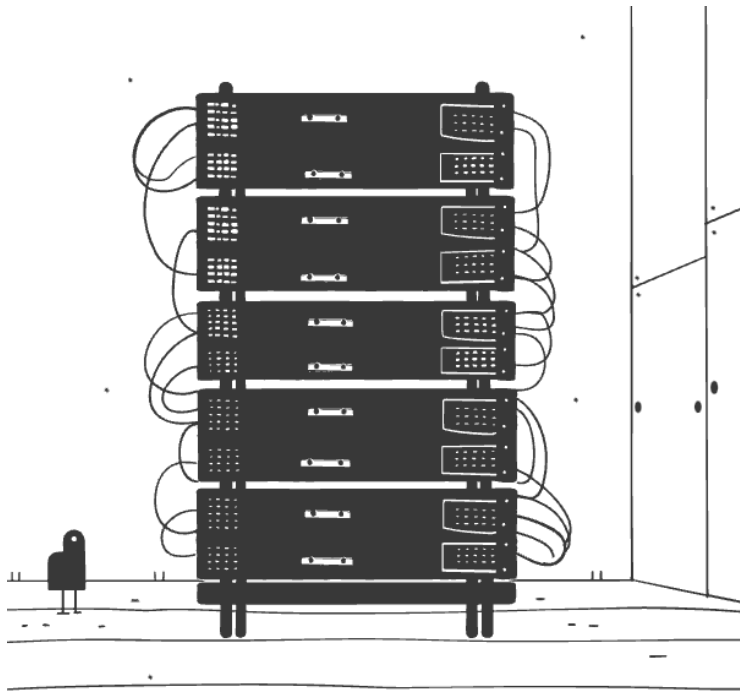
Can the system sustain target throughput without error rate climbing?



Soak (Endurance)

Are there memory leaks or connection pool exhaustion that only appear over hours?

Ramp Down — The Forgotten Profile



What It Reveals

Are connections, threads, and memory released cleanly after load drops?



Skipping ramp-down hides resource leak bugs that only surface under sustained production traffic.

Gatling Pattern

```
rampUsersPerSec(100).to(0).during(30)
```

Gatling: Full Load Profile in Code

Compose ramp up, steady state, and ramp down in a single `setUp` block:

```
setUp(  
  scenario.inject(  
    rampUsers(100).during(60),           // ramp up  
    constantUsersPerSec(100).during(300), // steady state  
    rampUsersPerSec(100).to(0).during(30) // ramp down  
  )  
)
```

 This pattern defines a complete, repeatable load shape — suitable for CI baseline comparisons on every build.

Gatling: Load Profiles as Code

```
class ProductApiSimulation extends Simulation {  
  
  val httpProtocol = http  
    .baseUrl("http://localhost:3000")  
    .header("Authorization", "Bearer mytoken")  
  
  val searchScenario = scenario("Product Search")  
    .exec(  
      http("GET /api/products")  
        .get("/api/products?q=headphones")  
        .check(status.is(200))  
    )  
  
  setUp(  
    searchScenario.inject(  
      rampUsers(100).during(30)  
    )  
  ).protocols(httpProtocol)  
  .assertions(  
    global.responseTime.percentile(95).lt(500),  
    global.failedRequests.percent.lt(1)  
  )  
}
```

HTML Report Includes

- Active users over time
- Response time distribution (p75 / p95 / p99)
- Requests per second & error breakdown
- Pass/fail against defined assertions



The `assertions` block makes Gatling **CI-friendly** — exits non-zero when SLAs are breached.

Advanced API & Security Testing

From dual-use performance tests to OWASP vulnerabilities — the complete automation engineer's toolkit across 7 sessions.

COURSE WRAP-UP

Karate: Write Once, Run as Functional & Load Test

Functional API Test (Gherkin)

```
Feature: Product API

Scenario: Get product by ID
  Given url
    'http://localhost:3000/api/products/1'
  And header Authorization = 'Bearer ' + token
  When method GET
  Then status 200
  And match response.product.name ==
    'Wireless Noise-Cancelling Headphones'
  And match response.product.price == '#number'
```

Same Scenario as Load Test

```
Runner.builder()
  .path("classpath:products.feature")
  .parallel(20) // 20 concurrent threads
  .build();
```



No code changes required to run as a load test.

Karate vs. Gatling: When to Choose

Criterion	Karate	Gatling
Reuse API tests as load tests	 Yes	 No
Advanced load profiles (ramp, spike)	Basic	Full
Report detail & percentile breakdowns	Moderate	Excellent
CI integration with pass/fail assertions	 Yes	 Yes

OWASP Top 10: Vulnerabilities Every Tester Should Know

1

Broken Access Control

Can User A access User B's data? Test with different auth tokens.

2

Cryptographic Failures

Sensitive data over plain HTTP? Passwords stored as plain text?

3

Injection

Does the API sanitise malicious input in query params and payloads?

4

Insecure Design

Can business logic be abused? Negative quantities, price manipulation?

5

Security Misconfiguration

Default credentials active? Stack traces in error responses?

OWASP A06–A10

1

Vulnerable Components (A06)

Outdated libraries with known CVEs? Run `npm audit`.

2

Authentication Failures (A07)

Brute-force possible? Sessions invalidated on logout?

3

Software & Data Integrity (A08)

Unverified dependencies in CI/CD pipeline?

4

Logging & Monitoring (A09)

Failed logins logged? Alerts triggered on anomalies?

5

SSRF (A10)

Can the app fetch internal network resources on an attacker's behalf?

Security Checks an Automation Engineer Can Own

Auth & Authorisation

- Protected endpoints return `401` without a token
- Standard user gets `403` on admin-only endpoints
- Token cannot be reused after logout

Data Exposure

- Response contains only authorised fields
- No internal IDs, file paths, or stack traces in errors

Input Validation

- API rejects payloads with unexpected fields
- Large payloads handled gracefully — no crash or timeout
- Malicious strings in query params are sanitised

Rate Limiting

- API returns `429` after rapid repeated requests
- `X-RateLimit-Remaining` headers present and accurate



CI/CD Security Gates on Every Pull Request

Tool	Type	What It Scans
OWASP ZAP	DAST	Running API — active & passive
Snyk	SCA+DAST	Dependencies & code issues
Trivy	SCA	Container images & K8s manifests
Semgrep	SAST	Source code anti-patterns
npm audit	SCA	Node.js CVEs — built-in

Three Pipeline Gates



Block Merge

HIGH or CRITICAL
vulnerabilities introduced



DAST

Run on every staging
deployment



npm audit

Run on every pull request

Playwright Covers a Significant Attack Surface

```
test('standard user cannot create products',
  async ({ request }) => {
    const loginRes = await
request.post('/api/auth/login', {
  data: { email: 'standard_user@example.com',
    password: 'Password123!' }
});
const { token } = await loginRes.json();
const res = await request.post('/api/products', {
  headers: { Authorization: `Bearer ${token}` },
  data: { name: 'Unauthorised', price: 0.01 }
});
expect(res.status()).toBe(403);
});
```

OWASP Coverage — No New Tools

→ A01 Broken Access Control

Wrong role, accessing another user's resource

→ A03 Injection

Malicious strings in API payloads and URL params

→ A05 Misconfiguration

Stack traces in errors, default credentials

→ A07 Auth Failures

Expired, missing, or reused tokens after logout

The Complete Automation Stack

Layer	Tool	When to Use
UI End-to-End	Playwright (TypeScript)	Critical user journeys & regression
API — Node.js	Playwright request	Contract tests & hybrid API+UI flows
API — Java	Rest Assured	Java backend API suites
BDD	Playwright + Cucumber	Scenarios requiring business sign-off
Visual	Playwright snapshots	UI-heavy releases, design system changes
Performance	Gatling / Karate	Pre-release load & NFR validation
Security	OWASP ZAP, Snyk, npm audit	Every PR (deps), every release (DAST)

01

Right Layer

Not everything needs a UI test

02

API Preconditioning

Makes UI tests faster and more reliable

03

Mock vs. Real

Mocks for edge cases; real backends for integration

04

Complete CI

Security and performance gates are non-negotiable

05

Automation as Product

Needs ownership, maintenance, and refactoring



Three Questions Before You Leave

? Question 1

Which of the 7 sessions **changed how you think** about testing?

? Question 2

What is the **first thing you will automate** when you return to your team?

? Question 3

Where are the **biggest gaps** — no API tests, no performance in CI, no security scanning?

The only wrong answer: "I'll get to it when things slow down."